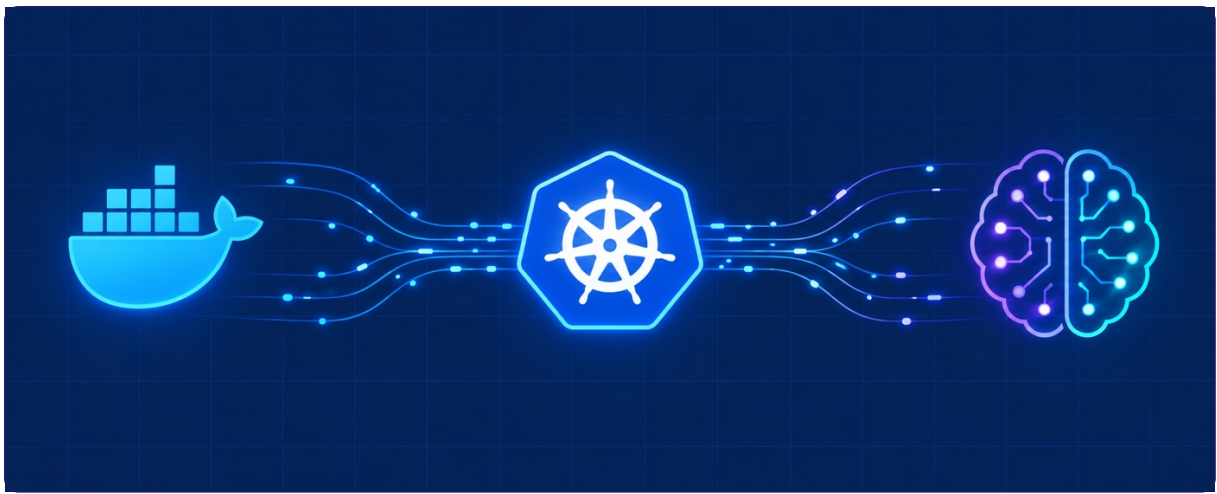




Rapport : Projet DevOps



Déploiement d'Ollama et Open WebUI

avec Docker Compose et Kubernetes (Minikube)

Étudiant

Abdoulaye Sy Ndaw

ndawas@ept.edu.sn

Professeur : Ibrahima MBENGUE

Dépôt : github.com/ndawas/llm-devops-project

Année académique : 2026

Table des matières

1 Introduction

Les modèles de langage (LLM) open source comme Llama 3, Mistral ou DeepSeek peuvent être exécutés localement, sans dépendre d'une API cloud payante. **Ollama** sert de moteur d'inférence local : il télécharge, charge en mémoire et sert ces modèles via une API HTTP sur le port 11434. **Open WebUI** fournit une interface de chat web qui consomme cette API, avec gestion de comptes utilisateurs et historique de conversation.

L'objectif de ce projet est de déployer cette plateforme selon deux approches :

1. **Docker Compose** : déploiement local rapide pour le développement, avec deux conteneurs (`ollama`, `open-webui`) définis dans un seul fichier `docker-compose.yaml`.
2. **Kubernetes (Minikube)** : déploiement orchestré simulant un environnement de production, avec neuf types de ressources définies dans `k8s/base/` et assemblées via Kustomize.

Ce rapport couvre cinq aspects techniques du déploiement Kubernetes : le rôle de chaque ressource, la communication entre services, la nécessité du stockage persistant, le rôle des probes, et la comparaison avec Docker Compose.

2 Rôle de chaque ressource Kubernetes

Le projet déploie deux applications (Ollama et Open WebUI) dans le namespace `llm-app`, à partir des manifests situés dans `k8s/base/`. Chaque ressource a un rôle précis dans le cycle de vie et l'exposition de ces applications.

2.1 Namespace

Le Namespace `llm-app` isole toutes les ressources du projet du reste du cluster Minikube. Toutes les ressources suivantes sont créées dans ce namespace, ce qui permet de gérer, lister et supprimer le projet entier sans toucher aux autres workloads du cluster.

```
apiVersion: v1
kind: Namespace
metadata:
  name: llm-app
```

2.2 ConfigMap

Deux ConfigMap stockent les variables de configuration en dehors des images Docker. `ollama-config` définit `OLLAMA_HOST=0.0.0.0` pour qu'Ollama écoute sur toutes les interfaces réseau du pod. `webui-config` définit `OLLAMA_BASE_URL=http://ollama-service:11434`, l'URL que Open WebUI utilise pour contacter Ollama. Les deux Deployments injectent ces valeurs via `envFrom.configMapRef`.

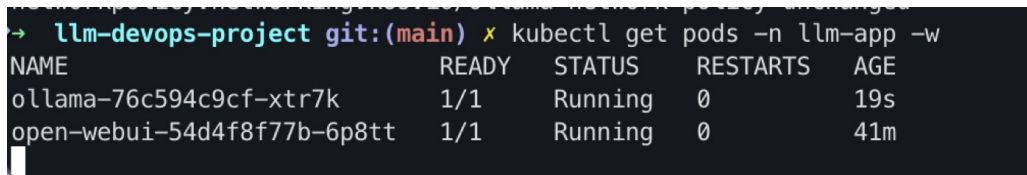
2.3 PersistentVolumeClaim

Deux PVC fournissent le stockage persistant : `ollama-pvc` (20Gi, monté sur `/root/.ollama`) pour les modèles LLM, et `webui-pvc` (2Gi, monté sur `/app/backend/data`) pour les comptes utilisateurs et l'historique de conversation d'Open WebUI. Le détail de leur nécessité est traité en section 4.

2.4 Deployment

Les Deployments `ollama` et `open-webui` gèrent le cycle de vie des pods : création, remplacement en cas de crash, rollout lors d'une mise à jour d'image. Chaque Deployment tourne avec

replicas: 1 dans ce projet, et définit les probes, les volumeMounts et les limites de ressources CPU/mémoire du conteneur.



```

→ llm-devops-project git:(main) ✗ kubectl get pods -n llm-app -w
NAME                                READY   STATUS    RESTARTS   AGE
ollama-76c594c9cf-xtr7k             1/1     Running   0           19s
open-webui-54d4f8f77b-6p8tt         1/1     Running   0           41m

```

FIGURE 1 – Pods Ollama et Open WebUI en cours d'exécution dans le namespace llm-app (kubectl get pods -n llm-app).

2.5 Service ClusterIP (ollama-service)

ollama-service est un Service de type ClusterIP qui expose le port 11434 des pods Ollama uniquement à l'intérieur du cluster. Il ne fournit pas d'IP externe accessible depuis l'extérieur de Minikube, ce qui correspond à son rôle : point d'entrée interne pour Open WebUI.

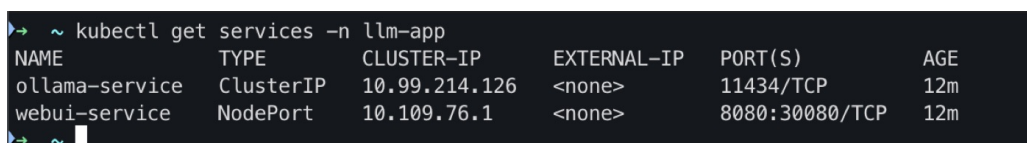
```

apiVersion: v1
kind: Service
metadata:
  name: ollama-service
spec:
  selector:
    app: ollama
  ports:
    - port: 11434
      targetPort: 11434
  type: ClusterIP

```

2.6 Service NodePort (webui-service)

webui-service expose le port 8080 du pod Open WebUI sur le port 30080 de chaque nœud du cluster, via **type: NodePort**. C'est ce qui permet d'accéder à l'interface web depuis l'extérieur de Minikube, par exemple avec `minikube service webui-service -n llm-app` ou directement sur `<node-ip>:30080`.



```

→ ~ kubectl get services -n llm-app
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
ollama-service      ClusterIP     10.99.214.126 <none>         11434/TCP        12m
webui-service       NodePort      10.109.76.1   <none>         8080:30080/TCP   12m

```

FIGURE 2 – Services ollama-service (ClusterIP) et webui-service (NodePort, port 30080) dans le namespace llm-app.

2.7 NetworkPolicy

ollama-network-policy restreint le trafic entrant vers les pods labellisés **app: ollama**. Seuls les pods labellisés **app: open-webui** peuvent atteindre le port 11434. Tout autre pod du cluster, même dans le même namespace, ne peut pas contacter Ollama directement. C'est une mesure de sécurité réseau au niveau du cluster, indépendante de toute authentification applicative.

2.8 PodDisruptionBudget

Deux PDB (`ollama-pdb` et `webui-pdb`) fixent `minAvailable: 1` pour chaque application. Lors d'une opération de maintenance planifiée sur le cluster (drain d'un nœud, mise à jour Kubernetes), l'API Kubernetes empêche l'éviction d'un pod si cela fait passer le nombre de pods disponibles sous ce seuil. Avec un seul replica par Deployment dans ce projet, le PDB garantit surtout que le scheduler ne vide pas le pod sans réflexion pendant une opération de cluster.

3 Communication entre les services

Le flux de communication suit un chemin fixe, défini par le DNS interne de Kubernetes et la NetworkPolicy.

1. Open WebUI lit la variable `OLLAMA_BASE_URL` (injectée depuis `webui-config`) et envoie ses requêtes API vers `http://ollama-service:11434`.
2. Le CoreDNS du cluster résout le nom `ollama-service` (format court, car les deux pods sont dans le même namespace `llm-app`) vers l'IP virtuelle (ClusterIP) attribuée au Service `ollama-service`.
3. Le kube-proxy route ce trafic vers un des pods backend du Service, sélectionnés via le label `app: ollama`, sur le port cible 11434.
4. La NetworkPolicy `ollama-network-policy` vérifie que la requête provient bien d'un pod avec le label `app: open-webui`. C'est le seul flux entrant autorisé vers Ollama sur ce port ; tout autre trafic est bloqué au niveau du plugin réseau CNI.
5. Côté utilisateur, l'accès se fait sur le NodePort 30080 exposé par `webui-service`. C'est le seul point d'entrée externe du système : Ollama n'est jamais exposé en dehors du cluster.

Flux de communication

Utilisateur $\xrightarrow{\text{NodePort 30080}}$ `webui-service` $\rightarrow$ pod `open-webui` $\xrightarrow{\text{DNS interne : ollama-service}}$ `ollama-service` (ClusterIP, filtré par NetworkPolicy) $\rightarrow$ pod `ollama` (port 11434).

Ce découpage sépare clairement le trafic interne (Open WebUI vers Ollama, contrôlé par la NetworkPolicy) du trafic externe (utilisateur vers Open WebUI, via NodePort).

4 Pourquoi les PersistentVolumeClaims sont nécessaires

Un pod Kubernetes est éphémère par conception. Quand un pod est recréé (crash, rollout, éviction par le scheduler), son système de fichiers local repart de zéro à partir de l'image du conteneur. Sans volume persistant, tout ce qui a été écrit dans le conteneur disparaît.

Ollama télécharge et stocke ses modèles LLM dans `/root/.ollama`. Sur ce projet, trois modèles sont utilisés : llama3, mistral et deepseek. Ces modèles pèsent plusieurs gigaoctets chacun, ce qui justifie les 20Gi alloués au `ollama-pvc`. Sans ce PVC, chaque redémarrage du pod Ollama forcerait un nouveau téléchargement complet des trois modèles avant de pouvoir répondre à la moindre requête, ce qui rend le service indisponible pendant plusieurs minutes à chaque redéploiement.

Open WebUI stocke les comptes utilisateurs, les paramètres et l'historique des conversations dans `/app/backend/data`, monté sur `webui-pvc` (2Gi). Sans ce PVC, un simple rollout du Deployment `open-webui` effacerait tous les comptes et tout l'historique de chat.

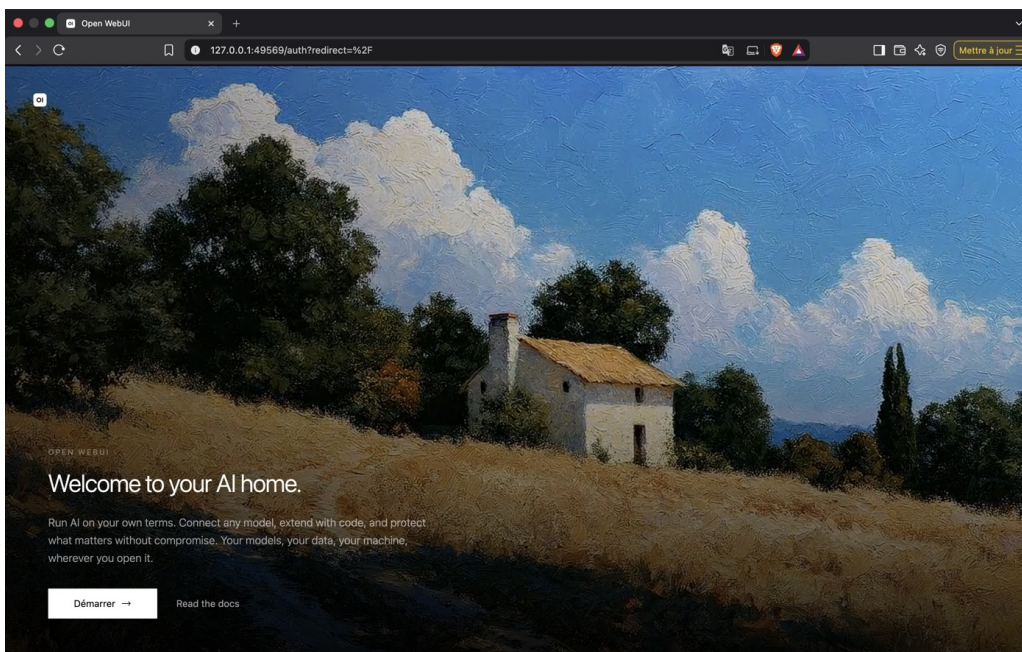


FIGURE 3 – Interface Open WebUI accessible depuis l'extérieur du cluster via le NodePort 30080, après résolution du flux interne vers `ollama-service`.

Avec les PVC en place, le stockage est découplé du cycle de vie du pod. Kubernetes réattache le même volume au nouveau pod après un redémarrage ou une mise à jour d'image, et les données (modèles téléchargés, comptes, historique) survivent à l'opération.

Les captures suivantes montrent deux des modèles répondant via l'instance Open WebUI exposée par Kubernetes (`minikube service webui-service`, tunnel `127.0.0.1:58018`), tous deux chargés depuis le volume persistant `ollama-pvc`.

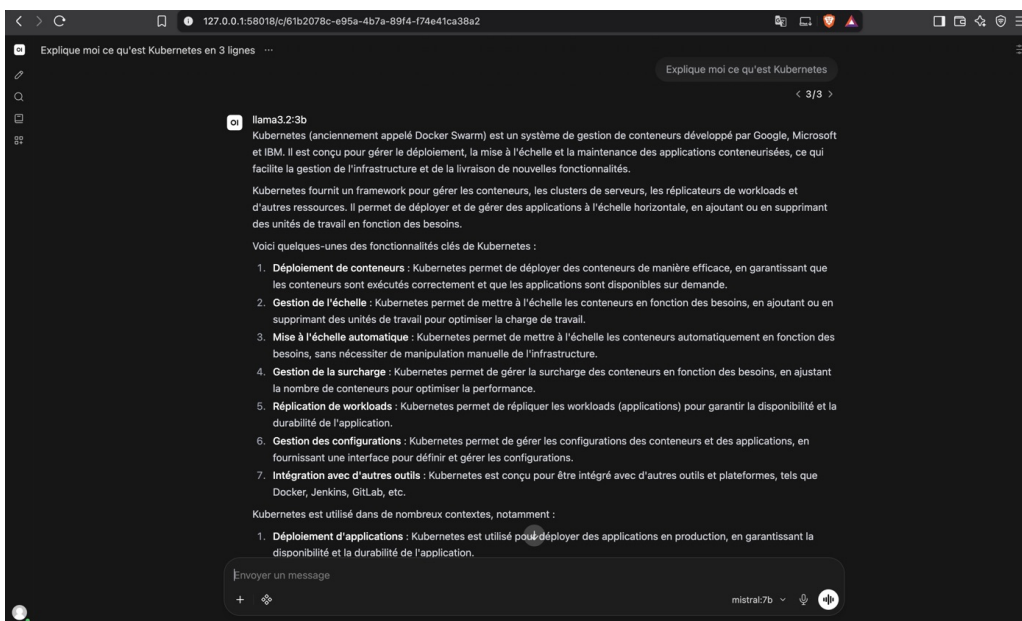


FIGURE 4 – Réponse de `llama3.2:3b` à la question « Explique moi ce qu'est Kubernetes », servie par le pod Ollama sur Kubernetes.

Le contenu technique est correct sur les fonctionnalités clés de Kubernetes (déploiement, scaling, réplication). Le temps de réponse observé confirme que le pod, une fois passé le `startupProbe`, sert les requêtes normalement via `ollama-service`.

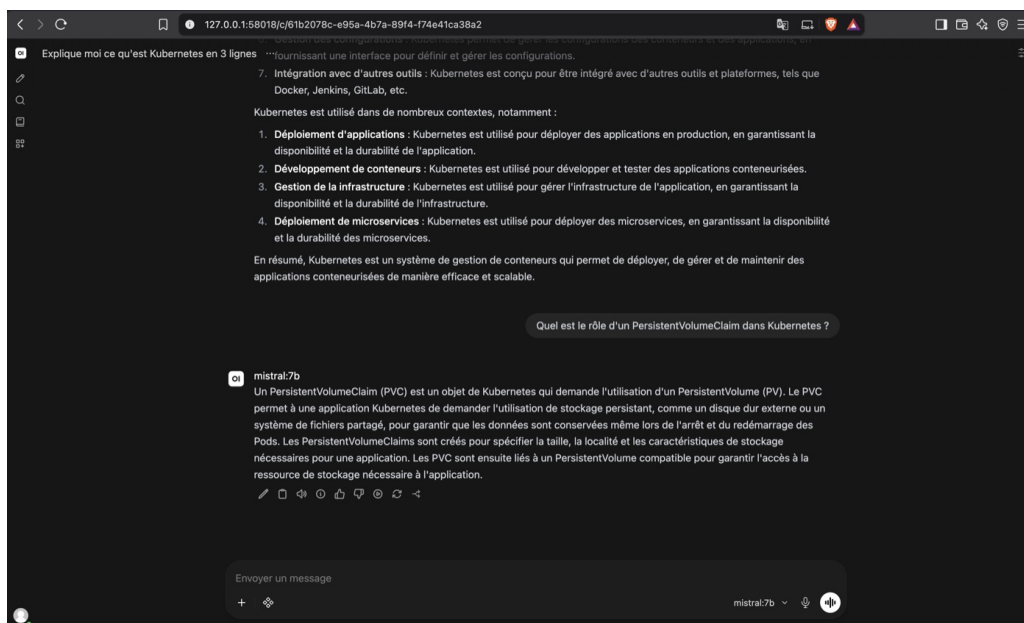


FIGURE 5 – Réponse de `mistral:7b` à la question « Quel est le rôle d'un PersistentVolumeClaim dans Kubernetes ? », servie par le pod Ollama sur Kubernetes.

La réponse est plus concise et plus précise que celle de `llama3.2:3b` sur cette question technique ciblée, ce qui est cohérent avec un modèle 7B mieux doté pour ce type d'explication factuelle. Elle mentionne correctement le lien entre PVC et PersistentVolume ainsi que la persistance des données au redémarrage des pods, exactement le point développé plus haut dans cette section.

5 Rôle des probes

Les Deployments définissent chacun trois types de probes, avec des mécanismes différents selon la nature de l'application.

5.1 `startupProbe`

Vérifie qu'Ollama a fini de démarrer avant que les autres probes ne prennent le relais. Configurée avec `failureThreshold: 30` et `periodSeconds: 10`, ce qui laisse jusqu'à 5 minutes (30 tentatives × 10 secondes) au conteneur pour devenir opérationnel avant que Kubernetes ne le considère en échec.

```
startupProbe:
  exec:
    command: ["ollama", "list"]
  failureThreshold: 30
  periodSeconds: 10
  timeoutSeconds: 10
```

5.2 readinessProbe

Détermine si le pod est prêt à recevoir du trafic via le Service. Tant que `ollama list` échoue, le pod est retiré des endpoints de `ollama-service`, et Open WebUI ne peut pas lui envoyer de requêtes. Cela évite d'envoyer des prompts vers une instance Ollama pas encore prête à les traiter.

5.3 livenessProbe

Détecte un pod bloqué (deadlock, process figé) qui répond toujours aux vérifications TCP de base mais n'exécute plus rien correctement. Si `ollama list` échoue de façon répétée après le démarrage, kubelet tue et redémarre le conteneur automatiquement, sans intervention manuelle.

5.4 timeoutSeconds

Fixé à 10 secondes sur toutes les probes d'Ollama, au lieu de la valeur par défaut de 1 seconde. La commande `ollama list` interroge le moteur d'inférence et peut prendre plus d'une seconde à répondre sur les ressources CPU limitées d'un environnement Minikube local, surtout quand un modèle est en cours de chargement en mémoire. Avec le timeout par défaut, les probes auraient échoué en continu par faux positifs.

Différence `exec` / `httpGet`

Ollama : probes en `exec` (`ollama list`), car c'est un moteur CLI sans endpoint HTTP dédié à la santé.

Open WebUI : probes en `httpGet` sur `/health`, port 8080, car c'est une application web qui expose nativement un endpoint HTTP de santé, plus léger qu'un `exec` (pas de fork de process à chaque probe).

5.5 Cas d'Open WebUI

Les probes d'`open-webui` utilisent `httpGet` sur le chemin `/health` et le port 8080, au lieu d'un `exec`. Contrairement à Ollama, Open WebUI est une application web qui expose nativement un endpoint HTTP de santé, et cela correspond directement au protocole que l'application sert déjà.



FIGURE 6 – Exposition de `webui-service` via `minikube service` : le tunnel ne s'ouvre que parce que la `readinessProbe` a validé le pod Open WebUI comme prêt à recevoir du trafic.

6 Différence entre Docker Compose et Kubernetes

6.1 Docker Compose

Docker Compose est un outil local orienté développement. Le fichier `docker-compose.yaml` de ce projet définit les deux services (`ollama`, `open-webui`), leurs volumes, leur réseau bridge et leurs `healthcheck` en une trentaine de lignes. Le démarrage se fait avec une seule commande :

```
docker compose up -d
```

Il n'y a ni haute disponibilité, ni self-healing au niveau orchestrateur : le `healthcheck` du conteneur `ollama` permet à `open-webui` d'attendre qu'Ollama soit `service_healthy` avant de démarrer (via `depends_on`), mais si le conteneur `ollama` crashe de façon répétée, Docker Compose ne fait que le relancer selon la police `restart: unless-stopped`, sans logique de scheduling, de scaling ou de répartition sur plusieurs hôtes.

```
llm-devops-project git:(main) # docker compose up -d
WARN[0000] /Users/simply_ablave/DIC2/Cloud-DevOps/dic2-2026/llm-devops-project/docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 30/30
✓ Image ollama/ollama:latest Pulled 547.0s
✓ Image ghcr.io/open-webui/open-webui:main Pulled 482.2s
✓ Network llm-devops-project_llm Created 0.1s
✓ Volume llm-devops-project_open-webui Created 0.0s
✓ Volume llm-devops-project_ollama Created 0.0s
✓ Container ollama Healthy 17.0s
✓ Container open-webui Started 16.9s
llm-devops-project git:(main) # docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
7c76482dca97   ghcr.io/open-webui/open-webui:main  "bash start.sh"         18 hours ago  Up 18 hours  0.0.0.0:3000->8080/tcp, [::]:3000->8080/tcp  open-webui
684c2928f994   ollama/ollama:latest                "/bin/ollama serve"     18 hours ago  Up 18 hours  0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp  ollama
```

FIGURE 7 – Conteneurs `ollama` et `open-webui` lancés localement via Docker Compose (`docker ps`).

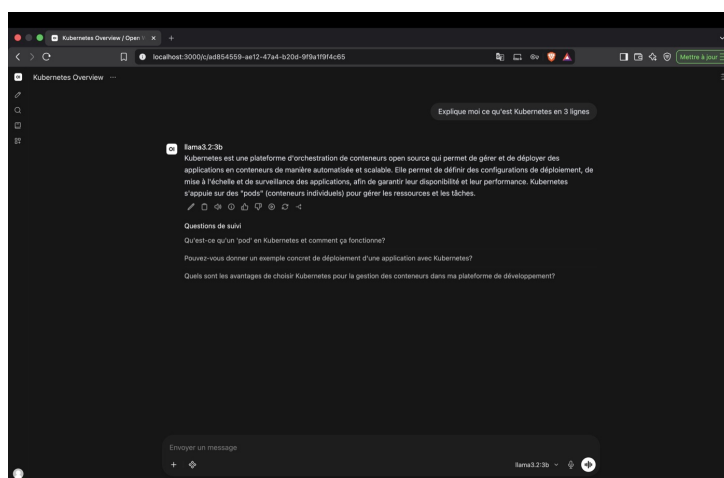


FIGURE 8 – `llama3` testé via l'instance Docker Compose locale (`localhost:3000`), avant déploiement sur Kubernetes.

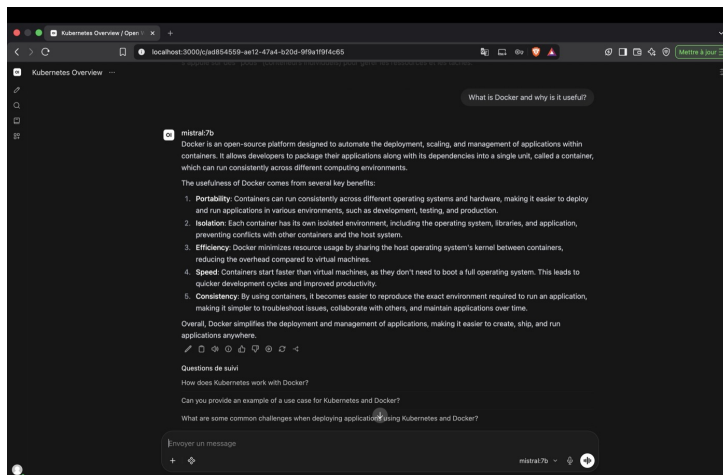


FIGURE 9 – mistral testé via l'instance Docker Compose locale.

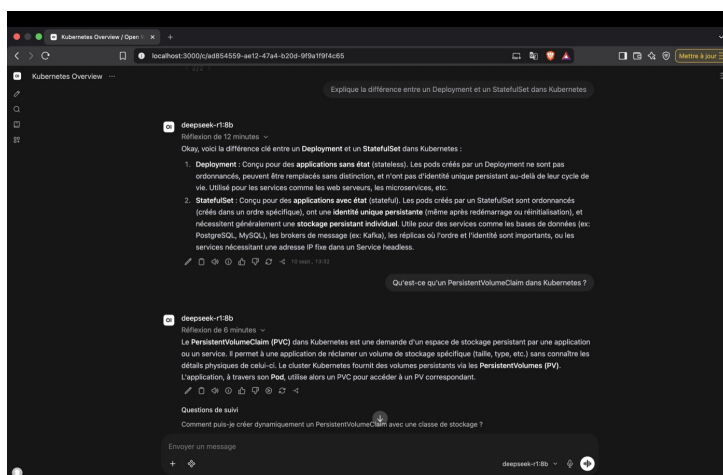


FIGURE 10 – deepseek testé via l'instance Docker Compose locale.

6.2 Kubernetes

Kubernetes ajoute une couche d'orchestration complète : self-healing via les probes et les Deployment controllers, scaling horizontal (non utilisé ici avec `replicas: 1`, mais disponible nativement), gestion fine des ressources CPU/mémoire par conteneur (`requests/limits`), health checks à trois niveaux (startup, readiness, liveness) contre un seul `healthcheck` par service dans Compose, stockage persistant natif découplé du pod via PVC, et NetworkPolicy pour isoler le trafic entre applications.

Critère	Docker Compose	Kubernetes
Périmètre	Machine locale unique	Cluster multi-nœuds
Self-healing	Redémarrage simple (<code>restart</code>)	Deployment controller + probes
Health checks	1 niveau (<code>healthcheck</code>)	3 niveaux (startup/readiness/liveness)
Scaling	Manuel, un seul hôte	Horizontal, natif
Stockage	Volumes Docker locaux	PVC, découplé du pod
Réseau	Réseau bridge simple	Services + NetworkPolicy

TABLE 1 – Comparaison fonctionnelle entre Docker Compose et Kubernetes sur ce projet.

7 Fonctionnalités bonus

Au-delà des exigences de base, ce projet implémente l'ensemble des sept fonctionnalités bonus proposées : NetworkPolicy et PodDisruptionBudget (détaillées en sections 2.7 et 2.8), HorizontalPodAutoscaler, overlays Kustomize dev/prod, chart Helm, pipeline CI/CD de déploiement et monitoring Prometheus/Grafana.

7.1 HorizontalPodAutoscaler

`webui-hpa` met à l'échelle automatiquement le Deployment `open-webui` entre 1 et 3 replicas, sur deux métriques : utilisation CPU moyenne à 70% et mémoire à 80% des `requests`. Le contrôleur interroge ces métriques via `metrics-server`, qui expose l'API `metrics.k8s.io` consommée par le HPA.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webui-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: open-webui
  minReplicas: 1
  maxReplicas: 3
  metrics:
    - type: Resource
      resource: {name: cpu, target: {type: Utilization,
        averageUtilization: 70}}
    - type: Resource
      resource: {name: memory, target: {type: Utilization,
        averageUtilization: 80}}
```

7.2 Overlays Kustomize (dev/prod)

`k8s/overlays/dev` et `k8s/overlays/prod` référencent `k8s/base` via Kustomize sans dupliquer les manifests, ce qui permet de faire varier la configuration par environnement (par exemple les limites de ressources ou le nombre de replicas) tout en gardant une seule source de vérité pour les ressources communes.

7.3 Chart Helm

Le répertoire `helm/llm-stack/` reproduit l'intégralité des ressources de `k8s/base` sous forme de chart Helm paramétrable via `values.yaml` (images, tailles de stockage, seuils de probes, activation de la NetworkPolicy et du PDB). Le chart est validé automatiquement :

```
$ helm lint helm/llm-stack
1 chart(s) linted, 0 chart(s) failed

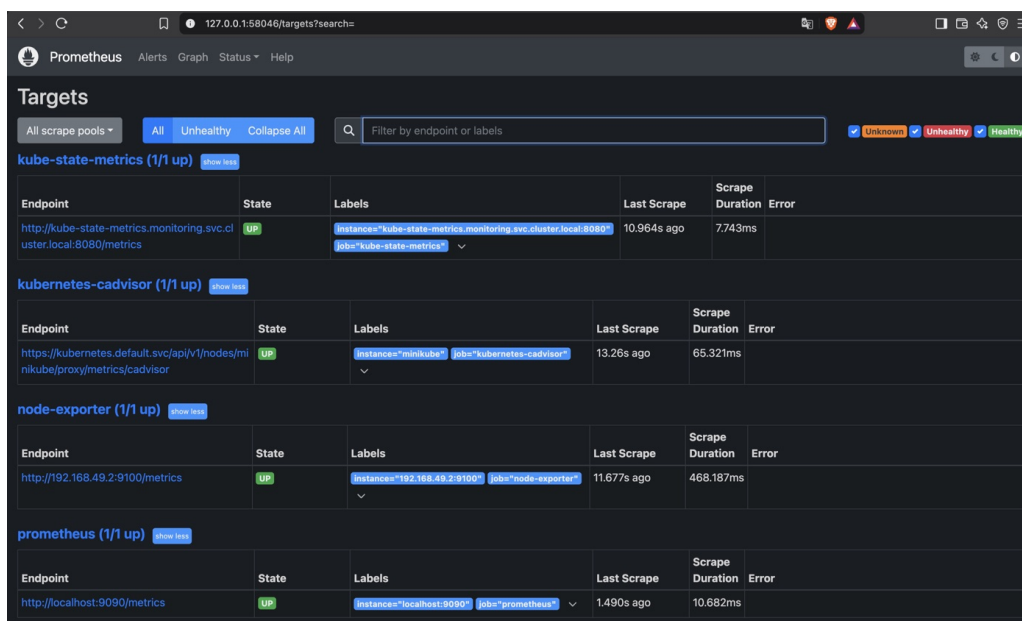
$ helm template llm-stack helm/llm-stack | kubeconform -strict -summary
-
Summary: 14 resources found parsing stdin - Valid: 14, Invalid: 0,
Errors: 0
```

7.4 Pipeline CI/CD de déploiement

Le workflow `.github/workflows/deploy.yml` ajoute un job `deploy` qui exécute `helm upgrade -install` sur un cluster distant. Il est gardé par une variable de dépôt (`CLUSTER_DEPLOY_ENABLED`) et un secret `KUBECONFIG_B64` : sans ces deux éléments configurés, le job est ignoré plutôt que d'échouer, ce qui est le comportement attendu tant que le cluster Minikube local n'est pas exposé à GitHub Actions.

7.5 Monitoring Prometheus et Grafana

Une stack de supervision complète est déployée dans le namespace `monitoring` : Prometheus (avec RBAC pour scraper les nœuds, pods et l'API `cadvisor`), kube-state-metrics (état des objets Kubernetes), node-exporter (métriques système) et Grafana, avec un datasource et un dashboard provisionnés automatiquement au démarrage.



Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
kube-state-metrics (1/1 up)					
http://kube-state-metrics.monitoring.svc.cluster.local:8080/metrics	UP	instances="kube-state-metrics.monitoring.svc.cluster.local:8080" job="kube-state-metrics"	10.964s ago	7.743ms	
kubernetes-cadvisor (1/1 up)					
https://kubernetes.default.svc/api/v1/nodes/minikube/proxy/metrics/cadvisor	UP	instances="minikube" job="kubernetes-cadvisor"	13.26s ago	65.321ms	
node-exporter (1/1 up)					
http://192.168.49.2:9100/metrics	UP	instances="192.168.49.2:9100" job="node-exporter"	11.677s ago	468.187ms	
prometheus (1/1 up)					
http://localhost:9090/metrics	UP	instances="localhost:9090" job="prometheus"	1.490s ago	10.682ms	

FIGURE 11 – Les 4 cibles Prometheus (prometheus, kube-state-metrics, kubernetes-cadvisor, node-exporter) sont toutes UP, sans erreur de scrape.

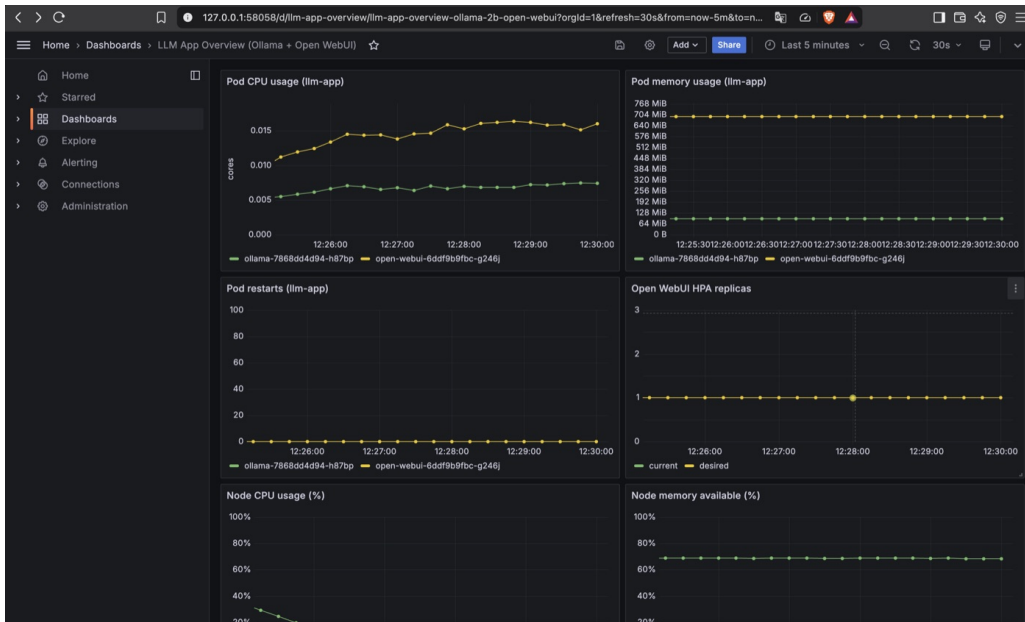


FIGURE 12 – Dashboard Grafana « LLM App Overview », haut de page : CPU et mémoire par pod (axe en cores), restarts (minimum à 0), replicas du HPA.

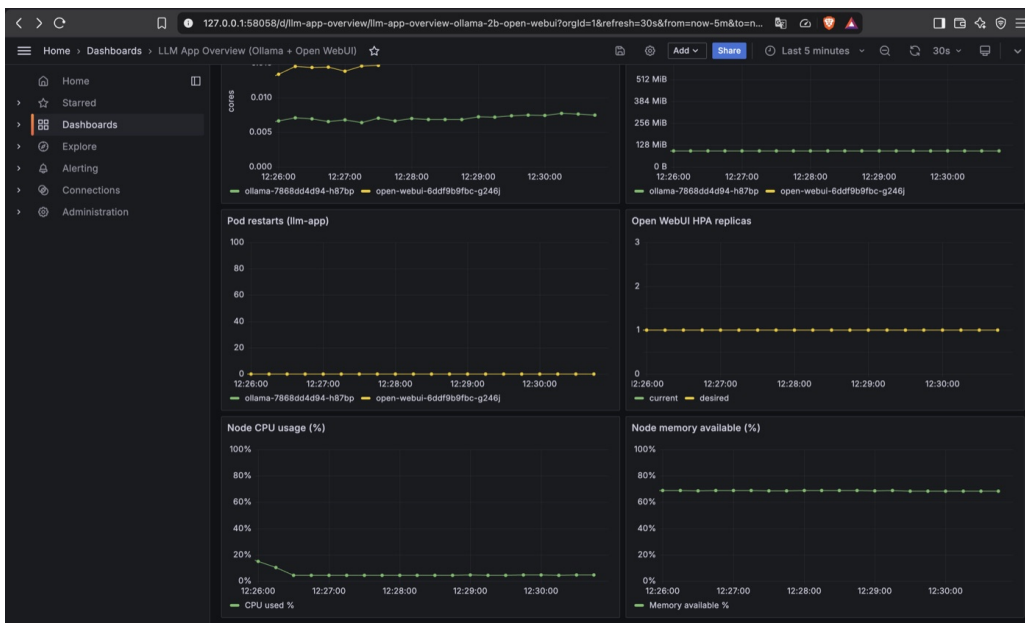


FIGURE 13 – Dashboard Grafana « LLM App Overview », suite : replicas du HPA en escalier (plafond à 3), CPU et mémoire du nœud Minikube.

Ce dashboard confirme en direct que les pods `ollama` et `open-webui` tournent sans redémarrage, que le HPA reste à 1 replica en charge normale, et que le nœud Minikube dispose de marge CPU et mémoire.

8 Conclusion

Ce projet a permis de déployer une plateforme de LLM auto-hébergée (Ollama + Open WebUI) selon deux approches complémentaires. Docker Compose a servi de base pour le développement local, avec un fichier unique et un démarrage en une commande. Kubernetes, via Minikube, a permis de simuler un environnement de production plus proche des contraintes réelles : isolation par namespace, configuration externalisée via ConfigMap, stockage persistant pour les modèles et les données utilisateurs, exposition contrôlée via Services ClusterIP et NodePort, restriction du trafic via NetworkPolicy, health checks à trois niveaux pour la résilience, et PodDisruptionBudget pour les opérations de maintenance. Les deux fichiers (`docker-compose.yaml` et `k8s/base/`) coexistent dans le dépôt et couvrent chacun une étape du cycle de vie du projet, complétés par un chart Helm, un pipeline de déploiement et une stack de monitoring pour les fonctionnalités bonus.

La principale leçon technique de ce projet est que l'orchestration Kubernetes n'ajoute pas seulement de la complexité : elle répond à des besoins concrets absents de Docker Compose, en particulier la persistance des données face à des pods éphémères et la détection automatique des pannes via les probes, deux points critiques pour un service qui manipule des modèles LLM volumineux et des données utilisateurs sensibles.

Références

1. Documentation officielle Kubernetes : <https://kubernetes.io/docs/>
2. Documentation officielle Docker Compose : <https://docs.docker.com/compose/>
3. Documentation Ollama : <https://github.com/ollama/ollama>
4. Documentation Open WebUI : <https://docs.openwebui.com/>
5. Documentation Minikube : <https://minikube.sigs.k8s.io/docs/>
6. Dépôt du projet : <https://github.com/ndawas/llm-devops-project>